

# CoreSight

AI-Powered Medical Diagnostic Platform

Pancreatic Tumor Detection via CT Scan Segmentation

|           |                                           |
|-----------|-------------------------------------------|
| Domain    | Frontend Dashboard + AI/ML Backend        |
| Framework | Next.js 16 / FastAPI / PyTorch            |
| Model     | True U-Net Semantic Segmentation          |
| Status    | Research Prototype   Not Production-Ready |

Prepared for Presentation Use | March 2026

## Project Overview

CoreSight is a full-stack AI-powered medical diagnostic web application designed to detect pancreatic tumors from abdominal CT scan images. It combines a modern clinical dashboard built with React/Next.js and a deep-learning Python backend running a True U-Net Segmentation model for pixel-level pancreas detection.

The platform targets hospital-grade clinical workflows: radiologists upload CT scans and instantly receive AI-generated diagnoses with bounding box overlays, red mask segmentation maps, and confidence metrics, all within a single authenticated web interface.

|                              |                          |                  |               |
|------------------------------|--------------------------|------------------|---------------|
| Hospital-Grade Clinical Tool | Pixel-Level Segmentation | ~600ms Inference | Firebase Auth |
|------------------------------|--------------------------|------------------|---------------|

## System Architecture at a Glance

| Layer       | Technology            | Role                                    |
|-------------|-----------------------|-----------------------------------------|
| Frontend    | Next.js 16 / React 19 | Clinical Dashboard & CT Upload UI       |
| Auth        | Firebase SDK 12       | Secure Login & Patient Sessions         |
| API Bridge  | Next.js API Routes    | Proxy between Frontend & AI Backend     |
| AI Backend  | FastAPI + Uvicorn     | REST Inference Endpoint                 |
| ML Model    | PyTorch U-Net         | Pixel-Level Segmentation (~7.8M params) |
| Image Proc. | OpenCV + Pillow       | ROI Crop, Overlays, Base64 Export       |

## SECTION 01 | FRONTEND DEVELOPMENT

Clinical Dashboard | React / Next.js / TypeScript

### What the Frontend Does

The frontend is the primary interface that radiologists and medical staff interact with. It handles user authentication, CT scan uploads, real-time AI result display with visual overlays, and historical diagnosis tracking, all within a responsive dark/light-mode dashboard.

### Core Pages & Modules

| Route / Module               | Purpose                                                    |
|------------------------------|------------------------------------------------------------|
| /login & /register           | Firebase-authenticated user onboarding                     |
| /dashboard                   | Protected overview with Recharts diagnostic history charts |
| /pancreas-scan               | CT Upload UI + live AI diagnosis card + heatmap viewer     |
| /diagnosis                   | Full patient diagnosis result history                      |
| /pancreas-info & /liver-info | Anatomical education pages with SVG illustrations          |
| SingleScanAnalysis.tsx       | Core component: upload, result card, overlay viewer        |

### Technology Stack -- Frontend

| Technology        | Version | Purpose                                           |
|-------------------|---------|---------------------------------------------------|
| Next.js           | 16.2.0  | Full-stack React framework with App Router        |
| React             | 19.2.3  | UI component library                              |
| TypeScript        | 5.x     | Type-safe development across all components       |
| TailwindCSS       | 4.x     | Utility-first responsive styling system           |
| Shadcn / Radix UI | 4.0.2   | Accessible headless component primitives          |
| Framer Motion     | 12.34   | Animated transitions and micro-interactions       |
| Lucide React      | 0.564   | Medical and clinical icon set                     |
| Recharts          | 3.8     | Diagnostic history charts and data visualizations |
| next-themes       | 0.4.6   | System-aware Dark / Light mode switching          |
| Firebase SDK      | 12.10   | Email/Password auth + Firestore patient database  |

### Key Frontend Features

### Firestore Authentication

Secure login and registration with persistent session management. All dashboard routes are protected and redirect unauthenticated users to /login automatically.

### CT Scan Upload Interface

The SingleScanAnalysis component handles file-picker CT uploads, sends images to the FastAPI backend via the Next.js API proxy, and displays the result card with confidence score and inference time.

### Real-Time Overlay Viewer

The AI's red translucent segmentation mask and green bounding box are decoded from the Base64 PNG in the API response and rendered as an overlay directly on the uploaded CT scan.

### Recharts Dashboard

Protected dashboard page plots historical diagnosis outcomes, confidence score trends, and inference timing using Recharts line and bar charts backed by Firestore.

### Dark / Light Mode

Full system-aware theme switching via next-themes with zero flash on load, togglable from the Navbar.

### Anatomical Info Pages

SVG-based digestive system illustrations for pancreas and liver anatomy -- useful for onboarding clinical staff unfamiliar with CT scan anatomy.

## How to Run the Frontend

Open PowerShell or terminal and run:

```
cd c:\projects\CoreSight
npm run dev
# Dashboard opens at http://localhost:3000
```

SECTION 02 | CORE ML MODEL & AI BACKEND

PyTorch U-Net | FastAPI | Medical Image Segmentation

### What the AI Backend Does

The AI backend is a self-contained Python system that receives a CT scan image via HTTP, validates it as a real abdominal CT, crops to the anatomical region of interest, runs it through a trained U-Net segmentation model, and returns a structured JSON response with a diagnosis label, confidence score, and Base64-encoded heatmap overlay in ~600ms.

### Technology Stack -- AI Backend

| Technology   | Purpose                                         |
|--------------|-------------------------------------------------|
| Python 3.13  | Backend runtime                                 |
| FastAPI      | Async REST API server (ASGI)                    |
| Uvicorn      | High-performance ASGI server                    |
| PyTorch      | Deep learning model training and inference      |
| TorchVision  | ImageNet normalization transforms               |
| nibabel      | NIfTI 3D medical volume loading (.nii.gz)       |
| NumPy        | Matrix operations and array processing          |
| OpenCV (cv2) | ROI cropping, contour detection, bounding boxes |
| Pillow       | Image format conversion and resizing            |
| tqdm         | Training progress bar visualization             |

### Dataset & Training Infrastructure

| Resource           | Details                                                       |
|--------------------|---------------------------------------------------------------|
| Primary Dataset    | Medical Segmentation Decathlon -- Task07 Pancreas             |
| Dataset Source     | Memorial Sloan Kettering Cancer Center                        |
| NIfTI Volumes Used | 15 patients (281 available -- only 15 used currently)         |
| Extracted Slices   | ~529 positive pancreas slices + 35 negative background slices |
| Training Platform  | Google Colab (NVIDIA T4 GPU) -- 10 to 15 minutes total        |
| Local Fallback     | CPU -- approximately 20 minutes per epoch                     |
| Model Weights      | UNET_PIPELINE/true_UNET_MODEL.pth (GPU-trained on Colab)      |

# The Complete AI/ML Pipeline

|        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| STEP 1 | <p><b>Data Extraction (extract_task07_data.py)</b></p> <p>Loads raw 3D NIfTI CT volumes (.nii.gz) using nibabel. For each patient: applies soft-tissue CT windowing (HU range -100 to +240); corrects the 90-degree NIfTI orientation via np.rot90; extracts positive slices where pancreas mask sum &gt; 0; samples up to 15 negative slices per patient; exports matched PNG pairs to data/images/ and data/masks/.</p>                                                                                                                                                                                                                                    |
| STEP 2 | <p><b>U-Net Architecture (model.py)</b></p> <p>A Semantic Segmentation Encoder-Decoder with ~7.8M trainable parameters. Encoder: 3 downsampling blocks (MaxPool2d + DoubleConv). Bottleneck: 512-channel deep feature representation. Decoder: 3 upsampling blocks (ConvTranspose2d + skip connections from encoder). Output: 1-channel pixel probability logit map matching input spatial size.</p>                                                                                                                                                                                                                                                         |
| STEP 3 | <p><b>Training (train.py)</b></p> <p>Loss: DiceBCELoss (Binary Cross-Entropy + Dice IoU). Optimizer: Adam, lr=1e-4. Scheduler: ReduceLROnPlateau (patience=3, factor=0.5). Early Stopping: patience of 7 epochs. Augmentation: synchronous horizontal flipping on image and mask. Normalization: ImageNet mean/std for backbone compatibility.</p>                                                                                                                                                                                                                                                                                                           |
| STEP 4 | <p><b>Inference Pipeline (pancre_scan_api.py)</b></p> <p>1. OOD Check: colorfulness metric blocks non-CT color images (threshold &gt; 75). 2. Auto ROI Crop: OpenCV finds the largest abdominal contour and crops to it. 3. U-Net Forward Pass: pixel-probability mask via torch.sigmoid. 4. Binary Threshold: pixels above 0.30 probability = pancreas/tumor. 5. Noise Filter: more than 50 disconnected fragments = rejected as static noise. 6. Confidence Gate: max probability must exceed 0.45. 7. Mask Area Check: positive region must cover at least 0.5% of image. 8. Output: green bounding box + red translucent mask encoded as Base64 PNG.</p> |

## API Response Schema

The FastAPI endpoint returns the following JSON on every inference request:

```
{
  "diagnosis": "pancreatic_tumor | inconclusive",
  "confidence": 0.87,
  "entropy": 0.0,
  "inference_ms": 620.5,
  "positive_class": "pancreatic_tumor",
  "positive_threshold": 0.50,
  "warning": null,
  "heatmap_png_base64": ""
}
```

## How to Run the AI Backend

```
cd c:\projects\CoreSight\pancrescan\PancreScan
uvicorn pancre_scan_api:app --port 8000
# API endpoint is live at http://localhost:8000
```

## Retraining the Model

- Add patient NIfTI volumes to Task07\_Pancreas/imagesTr/ and labelsTr/
- Run extraction: python extract\_task07\_data.py
- Upload the data/ folder and Colab notebook to Google Colab
- Train on free T4 GPU -- approx. 10 to 15 minutes for 20+ epochs
- Download true\_unet\_model.pth and replace weights in unet\_pipeline/

## SECTION 03 | KNOWN LIMITATIONS & FIXES

*Honest Assessment | Research Prototype -- Not Production-Ready*

### Current Model Shortcomings

This is a fully transparent assessment of where CoreSight currently falls short. The system is a proof-of-concept research prototype only. Diagnostic outputs must not be used for real clinical decisions in its present state.

|                 |                                                                                                                                                                                                                                                             |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CRITICAL</b> | <b>Critically Small Training Dataset</b>                                                                                                                                                                                                                    |
| <b>Problem</b>  | Trained on only 15 patients out of 281 available, yielding ~564 matched 2D slice pairs. State-of-the-art pancreas segmentation needs 200+ patients. The model memorizes those 15 patients' visual style instead of learning generalizable pancreas anatomy. |
| <b>Fix</b>      | Extract and train on all 281 patients from the full Task07 dataset using Colab Pro GPU.                                                                                                                                                                     |
| <b>CRITICAL</b> | <b>Severely Under-Trained (Stopped at Epoch 2)</b>                                                                                                                                                                                                          |
| <b>Problem</b>  | CPU training took ~20 min/epoch so the run was cancelled after only 2 of 30 planned epochs. At epoch 2 a U-Net is still in near-random-weight phase with no meaningful tissue boundaries. Diagnoses are essentially random.                                 |
| <b>Fix</b>      | Let the full 30-epoch loop run uninterrupted on Google Colab GPU (15 to 20 min total).                                                                                                                                                                      |
| <b>CRITICAL</b> | <b>No Distinction Between Pancreas and Tumor</b>                                                                                                                                                                                                            |
| <b>Problem</b>  | The extraction script merges class 1 (pancreas) and class 2 (cancer) into one positive label. The model cannot tell a healthy pancreas from a cancerous one -- it only detects 'any pancreatic tissue is present'.                                          |
| <b>Fix</b>      | Use a 2-channel output U-Net (out_channels=2) or add a secondary classification head to distinguish healthy vs malignant tissue regions.                                                                                                                    |
| <b>HIGH</b>     | <b>15:1 Positive-to-Negative Class Imbalance</b>                                                                                                                                                                                                            |
| <b>Problem</b>  | Only 35 negative background images were used against 529 positive slices. This extreme imbalance biases the model toward always predicting 'something is here', causing false positives on non-pancreas abdominal regions.                                  |
| <b>Fix</b>      | Balance the dataset to 1:1 or 2:1 ratio using the built-in negative sampler already patched into extract_task07_data.py.                                                                                                                                    |

|         |                                                                                                                                                                                                                      |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MEDIUM  | <b>OOD Colorfulness Check Has False Negatives</b>                                                                                                                                                                    |
| Problem | The OOD check uses a colorfulness threshold of 75. Some false-color medical images (liver scans with artificial rendering) may still pass through and receive garbage confident bounding boxes on unrelated imagery. |
| Fix     | Replace the heuristic with a proper binary OOD classifier (fine-tuned ResNet) trained to distinguish abdominal CT vs non-CT.                                                                                         |

|         |                                                                                                                                                                                                                                               |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MEDIUM  | <b>No Formal Dice Score Benchmarking</b>                                                                                                                                                                                                      |
| Problem | The model has never been evaluated against ground-truth NIfTI masks with standard medical imaging metrics. There are no Dice Coefficient, Sensitivity, Specificity, or False Positive Rate figures. All quality checks have been visual only. |
| Fix     | Run a formal evaluation script computing Dice IoU between predicted masks and NIfTI ground-truth for at least 10 held-out patient volumes.                                                                                                    |

|         |                                                                                                                                                                                                                                                               |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MEDIUM  | <b>224px Resolution Bottleneck</b>                                                                                                                                                                                                                            |
| Problem | Clinical CT slices are natively 512x512 pixels. All images are resized to 224x224 before the U-Net -- a 4.9x spatial downsampling that destroys fine detail. Sub-centimetre micro-tumors and ductal lesions physically cannot be detected at this resolution. |
| Fix     | Increase input resolution to 512x512 (more GPU memory required) or implement a sliding-window inference approach over high-resolution crops.                                                                                                                  |

## Summary -- Problems & Severity

| Problem                          | Severity | Priority Fix                       |
|----------------------------------|----------|------------------------------------|
| Only 15 patients trained         | Critical | Extract all 281 patients, retrain  |
| Stopped at Epoch 2               | Critical | Full 30-epoch GPU run on Colab     |
| No pancreas vs tumor distinction | Critical | 2-channel U-Net output head        |
| 15:1 class imbalance             | High     | Balance negatives to 1:1 ratio     |
| OOD colorfulness heuristic       | Medium   | Train ResNet OOD classifier        |
| No Dice score benchmarking       | Medium   | Formal evaluation script           |
| 224px resolution limit           | Medium   | Upgrade to 512px or sliding-window |

**CoreSight is a research prototype with significant clinical potential.**

The foundation -- a full-stack clinical dashboard paired with a True U-Net segmentation backend -- is architecturally sound. Addressing the three critical gaps (dataset size, training epochs, and class distinction) would transform CoreSight into a genuinely useful screening aid. All three fixes are achievable within a single Google Colab session.